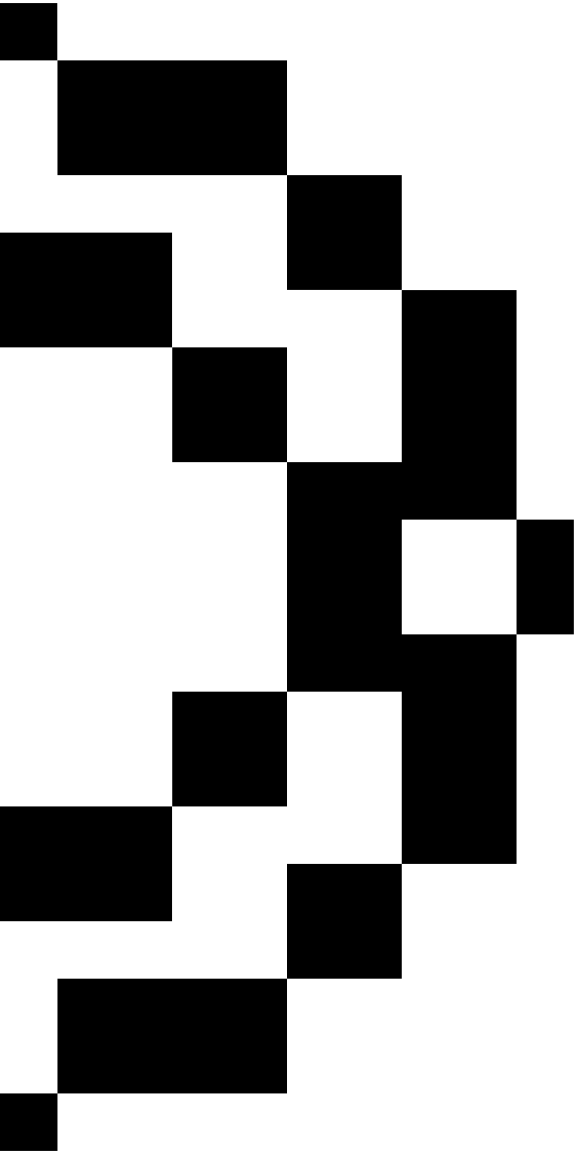




Adaptive Boosting (AdaBoost)

Leonardo Vanneschi

lvanneschi@novaims.unl.pt

NOVA Information Management School (NOVA IMS)
Universidade Nova de Lisboa
Campus de Campolide
1070-312 Lisboa, Portugal

What we have seen so far...

With Random Forests, we have seen one possible way of improving the performance of a single classifier: build many independent models and let them vote.

This philosophy has proved extremely successful. Random Forests are among the most widely used Machine Learning algorithms because they are usually accurate, robust to overfitting, and relatively easy to use. Their success stems from a very simple idea: different trees make different mistakes, and majority voting tends to eliminate many of those mistakes.

However, Random Forests are not the only way to build an ensemble. In fact, they belong to a broader family of methods called bagging algorithms, whose key idea is that all models are built independently of one another.

Towards Boosting

At this point, a question arises naturally, is independence (of the single models) really the best strategy?

Suppose that, after training the first classifier, we could look at the mistakes it made. Wouldn't it be reasonable to ask the second classifier to focus particularly on those difficult examples?

... And then ask the third classifier to focus on the mistakes made by the first two?

And so on...

Towards Boosting

In other words, instead of building many independent models and combining them at the end, could we build a sequence of models in which each new model learns from the mistakes of its predecessors?

This is precisely the philosophy behind boosting.

Boosting

Rather than training all learners independently, boosting constructs them sequentially.

Each learner is influenced by those that came before it, with the explicit goal of progressively correcting their errors.

The ensemble therefore evolves step by step, becoming increasingly specialized on the examples that are hardest to classify.

AdaBoost

Among all boosting algorithms, the most historically important (and arguably the simplest to understand) is AdaBoost, short for Adaptive Boosting.

Proposed by Y. Freund and R. Schapire, AdaBoost was one of the first ensemble methods to demonstrate that combining many weak learners could produce a highly accurate classifier.

AdaBoost is “Adaptive”

The word "adaptive" is the key to understanding the algorithm. Unlike bagging, where every training example has the same importance throughout the learning process, AdaBoost continuously adapts the learning procedure.

After each iteration, it increases the importance of the examples that were misclassified and decreases the importance of those that were correctly classified.

Consequently, each successive classifier concentrates its effort where the previous ones struggled the most.

A New Philosophy

Notice how different AdaBoost's philosophy is from Random Forests.

In Random Forests, diversity is obtained by exposing each tree to different subsets of the data and different subsets of the features.

In AdaBoost, diversity is obtained because each classifier is trained on a different distribution of the training examples.

As the algorithm progresses, the training data are effectively "*reshaped*": difficult examples receive increasingly larger weights, while easy examples gradually become less influential.

“Weak” Learners

This immediately suggests another important observation. Since each classifier is specifically designed to correct the mistakes of the previous ones, there is no need for each individual classifier to be extremely sophisticated.

In fact, AdaBoost is usually paired ***with very simple models***, often decision trees consisting of ***a single split***, known as ***decision stumps***.

Although each stump is only slightly better than random guessing, their sequential combination can produce an extremely powerful classifier.

A First High-Level Overview of AdaBoost

The AdaBoost algorithm can be summarized in the following steps:

1. Assign the same weight to every training example.
At the beginning, every example is considered equally important.
2. Train a weak learner.
Typically, this is a very simple classifier, such as a decision stump (a decision tree with a single split).
3. Evaluate its performance.
Identify which training examples have been classified correctly and which have been misclassified. (continues in the next slide...)

4. Update the weights of the training examples.

Increase the weights of the misclassified examples and decrease the weights of the correctly classified ones. As a result, the difficult examples become more important for the next iteration.

5. Train another weak learner.

The new learner is trained on the same dataset, but now it pays much more attention to the examples that previous learners found difficult.

6. Repeat the process.

Several weak learners are generated, each one focusing on correcting the mistakes of its predecessors.

(continues in the next slide...)

7. Combine all learners into a final classifier.

The final prediction is obtained through a weighted vote, where more accurate learners receive a larger influence on the final decision.

All these steps, that have now been presented in a high-level way, will be specified in detail by means of a step-by-step example.

A First High-Level Overview of AdaBoost

Although the previously presented procedure may seem straightforward, two important questions immediately arise:

- How exactly are the example weights updated after each iteration?
- How are the weights of the individual learners computed for the final weighted vote?

The best way to understand how all these ideas fit together is to work through a complete numerical example.

We shall now follow AdaBoost step by step, starting from a small training set and observing how the final ensemble is obtained.

A Step-by-Step Example

AdaBoost – A Step-by-Step Example

Let us consider the following dataset:

INPUT			TARGET
CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE
YES	YES	205	YES
NO	YES	180	YES
YES	NO	210	YES
YES	YES	167	YES
NO	YES	156	NO
NO	YES	125	NO
YES	NO	168	NO
YES	YES	172	NO

This is a binary classification dataset, whose objective is to allow prediction of **Heart Disease** using features such as **Chest Pain**, **Blocked Arteries** and **Patient Weight**.

We have 8 observations (patients) and one target (dependent variable).

First Step – Set Initial Sample Weights

The first step consists in initializing a new column that will establish the importance (**weight**) of each observation (*“how important it is that that observation is correctly classified”*).

During the development of the algorithm, the sample weights will get larger for observations that were misclassified by the weak learners.

Those observations will be considered by the algorithm harder to classify and so giving them more “importance” (more weight) will be our way to try to “push” the algorithm to classify them correctly.

First Step – Set Initial Sample Weights

In the beginning, we have no information on how the weak learners will behave on the single observations...

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	$1/8$
NO	YES	180	YES	$1/8$
YES	NO	210	YES	$1/8$
YES	YES	167	YES	$1/8$
NO	YES	156	NO	$1/8$
NO	YES	125	NO	$1/8$
YES	NO	168	NO	$1/8$
YES	YES	172	NO	$1/8$

... So we will give all observations the same weight:

$1 / \text{Total Number of Samples}$

Second Step – Create the First Weak Learner

In this example, we will consider that the weak learners are elementary decision trees, composed by just the root and two leaves (called decision stumps, or simply ***stumps***). As we said previously, although simple, this is a popular choice.

So, we need to choose the variable that will be at the root of our first stump...

... Although various ways to choose the root node exist, a good and very popular method is to use the **GINI Score**.

Second Step – Create the First Weak Learner

As we already know, the GINI Score (or GINI coefficient, or weighted GINI impurity) is a measure to quantify the ability of a variable to correctly separate samples.

The GINI Score value is by definition included in $[0, 1]$, where:

- 0 indicates that the variable is sufficient to perfectly separate all samples;
- 1 indicates that using that variable to classify is equivalent to a random guess.

On the Codomain of the GINI Score

About the interval of possible values of the Gini Score, it should be remarked that for binary classification, the Gini Score can actually take values only in $[0, 0.5]$.

When it comes to multiclass classification, the Gini Score takes values in $[0, w]$, where the bigger the number of classes, the bigger w ; and with w that tends to 1 with the number of classes growing.

Parentheis on the GINI Score

The GINI Score of a feature x is:

$$\text{GINI}(x) = \sum_{k=1}^N w_k G_k$$

where:

- N is the total number of possible values that x can take
- For each $k = 1, 2, \dots, N$: w_k is the probability that x takes the k^{th} possible value.

(... continues in the next slide with the definition of G_k).

Parentheis on the GINI Index

Also:

- for each $k = 1, 2, \dots, N$, the GINI Impurity G_k is defined as:

$$G_k = 1 - \sum_{j=1}^M p_{jk}^2$$

where:

- M is the total number of possible values that the target can take
- for all $j = 1, 2, \dots, M$: p_{jk} is the probability that x takes its k^{th} value AND that the target takes its j^{th} possible value.

The next examples should clarify this.

Back on our example...

Let us calculate the GINI Score of Chest Pain:

$P(\text{Chest Pain} = \text{YES} \ \& \ \text{Heart Disease} = \text{YES}) = 3/5$

$P(\text{Chest Pain} = \text{YES} \ \& \ \text{Heart Disease} = \text{NO}) = 2/5$

(notice that there are 5 observations with Chest Pain = YES in total).

So, the GINI impurity of the left branch, that we will indicate G_1 , is:

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	1/8
NO	YES	180	YES	1/8
YES	NO	210	YES	1/8
YES	YES	167	YES	1/8
NO	YES	156	NO	1/8
NO	YES	125	NO	1/8
YES	NO	168	NO	1/8
YES	YES	172	NO	1/8

$$G_1 = 1 - \left(\left(\frac{3}{5} \right)^2 + \left(\frac{2}{5} \right)^2 \right) = 0.48$$

Back on our example...

Also, we have:

$$P(\text{Chest Pain} = \text{NO} \ \& \ \text{Heart Disease} = \text{YES}) = 1/3$$

$$P(\text{Chest Pain} = \text{NO} \ \& \ \text{Heart Disease} = \text{NO}) = 2/3$$

So, the GINI impurity of the right branch, that we will indicate G_2 is:

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	1/8
NO	YES	180	YES	1/8
YES	NO	210	YES	1/8
YES	YES	167	YES	1/8
NO	YES	156	NO	1/8
NO	YES	125	NO	1/8
YES	NO	168	NO	1/8
YES	YES	172	NO	1/8

$$G_2 = 1 - \left(\left(\frac{1}{3} \right)^2 + \left(\frac{2}{3} \right)^2 \right) = 0.44$$

Second Step – Create the First Weak Learner

Now, the GINI Score of Chest Pain is:

$$\text{GINI}(\text{CHEST PAIN}) = w_1 G_1 + w_2 G_2$$

WHERE :

$$w_1 = P(\text{CHEST PAIN} = \text{YES}) = \frac{5}{8}$$

$$w_2 = P(\text{CHEST PAIN} = \text{NO}) = \frac{3}{8}$$

SO :

$$\text{GINI}(\text{CHEST PAIN}) = \frac{5}{8} \cdot 0.48 + \frac{3}{8} \cdot 0.44 = \underline{0.4667}$$

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	1/8
NO	YES	180	YES	1/8
YES	NO	210	YES	1/8
YES	YES	167	YES	1/8
NO	YES	156	NO	1/8
NO	YES	125	NO	1/8
YES	NO	168	NO	1/8
YES	YES	172	NO	1/8

Second Step – Create the First Weak Learner

Now let us calculate the GINI Score of Blocked Arteries (BA):

$$P(BA = YES \wedge HD = YES) = 3/6$$

$$P(BA = YES \wedge HD = NO) = 3/6$$

$$G_1 = 1 - \left(\left(\frac{3}{6} \right)^2 + \left(\frac{3}{6} \right)^2 \right) = 0.5$$

$$P(BA = NO \wedge HD = YES) = 1/2$$

$$P(BA = NO \wedge HD = NO) = 1/2$$

$$G_2 = 1 - \left(\left(\frac{1}{2} \right)^2 + \left(\frac{1}{2} \right)^2 \right) = 0.5$$

$$\begin{aligned} \text{GINI}(BA) &= P(BA = YES) \cdot 0,5 + P(BA = NO) \cdot 0,5 = \\ &= \frac{6}{8} \cdot 0,5 + \frac{2}{8} \cdot 0,5 = \underline{\underline{0.5}} \end{aligned}$$

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	1/8
NO	YES	180	YES	1/8
YES	NO	210	YES	1/8
YES	YES	167	YES	1/8
NO	YES	156	NO	1/8
NO	YES	125	NO	1/8
YES	NO	168	NO	1/8
YES	YES	172	NO	1/8

Second Step – Create the First Weak Learner

Now, let us finally calculate the GINI Score of (Patient Weight > 176) (let's call it PW):

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	1/8
NO	YES	180	YES	1/8
YES	NO	210	YES	1/8
YES	YES	167	YES	1/8
NO	YES	156	NO	1/8
NO	YES	125	NO	1/8
YES	NO	168	NO	1/8
YES	YES	172	NO	1/8

$$P(PW = YES \wedge HD = YES) = 3/3 = 1$$
$$P(PW = YES \wedge HD = NO) = 0/3 = 0$$
$$G_1 = 1 - (1^2 + 0^2) = 0$$

$$P(PW = NO \wedge HD = YES) = 1/5$$
$$P(PW = NO \wedge HD = NO) = 4/5$$

$$G_2 = 1 - \left(\left(\frac{1}{5} \right)^2 + \left(\frac{4}{5} \right)^2 \right) = 0.32$$

$$GINI(PW) = P(PW = YES) \cdot 0 + P(PW = NO) \cdot 0.32 =$$
$$= 0 + \frac{5}{8} \cdot 0.32 = \underline{0.2}$$

Second Step – Create the First Weak Learner

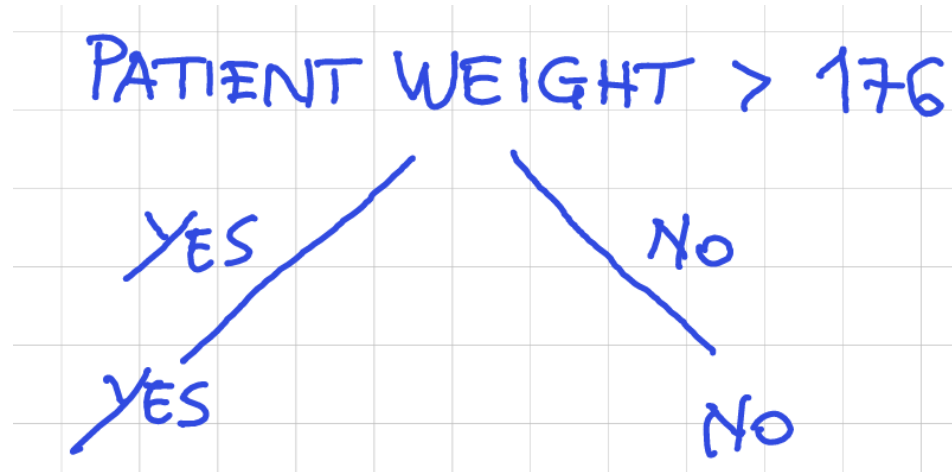
So, in the end, we have:

- $\text{GINI}(\text{Chest Pain}) = 0.4667$
- $\text{GINI}(\text{Blocked Arteries}) = 0.5$
- $\text{GINI}(\text{Patient Weight} > 176) = 0.2$

The root node with the **smallest** GINI index is (Patient Weight > 176).

Second Step – Create the First Weak Learner

So, finally best stump we have found (that will be our first weak learner) is:



Notice that so far we did not use the information coming from the sample weight.

This happened because all the samples had the same weight, so we simply ignored that information.

Third Step: Determine The Amount of Say

We now want to determine how much say the previous weak learner will have in the final classification.

This depends on how well the weak learner classifies the samples.

Third Step: Determine The Amount of Say

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	$1/8$
No	YES	180	YES	$1/8$
YES	No	210	YES	$1/8$
YES	YES	167	YES	$1/8$
No	YES	156	No	$1/8$
No	YES	125	No	$1/8$
YES	No	168	No	$1/8$
YES	YES	172	No	$1/8$

PATIENT WEIGHT > 176



THIS WEAK LEARNER MAKES ONE ERROR

IT MISCLASSIFIES

THIS PATIENT

Third Step: Determine The Amount of Say

The **total error** for a weak learner is the sum of the weights associated with the samples it incorrectly classifies.

Thus, in this case, the total error of the first weak learner is **1/8**.

Remark: given that the sample weights sum up to 1, the total error of a weak learner will always be between 0 (for a perfect learner) and 1 (for the learner that misclassifies all the samples).

Third Step: Determine The Amount of Say

To determine the amount of say of the weak learner, we use the following formula, based on its total error:

$$\text{AMOUNT OF SAY} = \frac{1}{2} \log \left(\frac{1 - \text{TOTAL ERROR}}{\text{TOTAL ERROR}} \right)$$

Justification for the Amount of Say

What AdaBoost actually does is a greedy minimization of the exponential loss:

$$L(F) = \sum_{i=1}^N \exp(-y_i F(x_i))$$

where:

- N is the number of training examples;
- x_i is the i^{th} training example;
- $y_i \in \{-1, 1\}$ is the class label of the i^{th} training example;
- $F(x)$ is the ensemble classifier, defined as

$$F(x) = \sum_{t=1}^T \alpha_t h_t(x)$$

with:

- T is the number of weak learners of which the ensemble is composed (n. of boosting iterations);
- $h_t(x) \in \{-1, 1\}$ the t^{th} weak learner (typically a decision stump);
- α_t is the amount of say of the t^{th} weak learner.

Justification for the Amount of Say

Substituting $F(x)$, the loss becomes:

$$L = \sum_{i=1}^N \exp \left(-y_i \sum_{t=1}^T \alpha_t h_t(x_i) \right).$$

Interpretation

The quantity $y_i F(x_i)$ is called the **margin**.

- If $y_i F(x_i) > 0$, the example is correctly classified.
- If $y_i F(x_i) < 0$, the example is misclassified.

Justification for the Amount of Say

The exponential loss behaves as follows:

- correctly classified examples receive a small penalty;
- misclassified examples receive an exponentially increasing penalty.

This is precisely why AdaBoost progressively focuses on the difficult training examples: after each iteration, the examples with larger exponential loss receive larger weights in the next boosting round.

Justification for the Amount of Say

Another way of interpreting the exponential loss is the following: let $\text{SNG}(y)$ be a function that returns $+1$ if $y \geq 0$, and -1 otherwise.

Given that both the true label y_i and the classifier output $\text{SNG}(F(x_i))$ are either -1 or $+1$, if they are identical then their product is $+1$, and so we sum up $\exp(-1)$ (~ 0.368).

Otherwise, if the true label and the classifier output are different, their product is -1 , and so we sum up $\exp(1)$ (~ 2.718).

Given that $\exp(1) > \exp(-1)$, the exponential loss $\mathcal{L}$ is minimum when all the classifier outputs are identical to the corresponding true labels.

Justification for the Amount of Say

Assume we already have an ensemble $F_{t-1}(x)$ and we add a new weak learner.

Now the ensemble is $F_t(x) = F_{t-1}(x) + \alpha h(x)$ and the loss becomes:

$$\mathcal{L} = \sum_i \exp \left(-\gamma_i \cdot \left(F_{t-1}(x_i) + \alpha h(x_i) \right) \right)$$

Justification for the Amount of Say

Let's develop the previous equation:

$$\begin{aligned}\mathcal{L} &= \sum_i \exp \left(-\gamma_i \cdot \left(F_{t-1}(x_i) + \alpha h(x_i) \right) \right) \\ &= \sum_i \exp \left(-\gamma_i F_{t-1}(x_i) - \alpha \gamma_i h(x_i) \right) \\ &= \sum_i \exp \left(-\gamma_i F_{t-1}(x_i) \right) \cdot \exp \left(-\alpha \gamma_i h(x_i) \right)\end{aligned}$$

Justification for the Amount of Say

Notice that, given that at time $t-1$ we had:

$$\mathcal{L} = \sum_i \exp(-y_i F_{t-1}(x_i))$$

Let's set:

$$w_i = \exp(-y_i F_{t-1}(x_i))$$

we will discover later that these are exactly the sample weights at time $t-1$.

Justification for the Amount of Say

Now, if the prediction of the new weak learner h is correct, then we have

$$y_i h(x_i) = +1$$

Otherwise, we have:

$$y_i h(x_i) = -1$$

Justification for the Amount of Say

So, we can rewrite the loss as follows:

$$\begin{aligned} \mathcal{L} &= \sum_i w_i \cdot \exp(-\alpha y_i h(x_i)) = \\ &= \sum_{\text{CORRECT}} w_i e^{-\alpha} + \sum_{\text{WRONG}} w_i e^{+\alpha} \end{aligned}$$

Let:

$$W_C = \sum_{\text{CORRECT}} w_i \qquad W_E = \sum_{\text{WRONG}} w_i$$

Justification for the Amount of Say

From the previous page, it is important to remark that:

- W_E is, for its own definition, the total error of the ensemble (i.e., the sum of the sample weights of the misclassified samples)
- Given that we always set the sample weights in such a way that they all sum up to 1:

$$W_C = 1 - W_E$$

Justification for the Amount of Say

Now, to find the α that minimizes $\mathcal{L}$, we have to find the α for which:

$$\frac{\delta \mathcal{L}}{\delta \alpha} = 0$$

Let's solve this equation:

$$\frac{\delta \mathcal{L}}{\delta \alpha} = \frac{\delta (w_c e^{-\alpha} + w_E e^{\alpha})}{\delta \alpha} = -w_c e^{-\alpha} + w_E e^{\alpha} = 0$$

$$[\text{REMINDER : IF } f(x) = e^x, \text{ THEN } f'(x) = e^x]$$

Justification for the Amount of Say

Continuing to solve the equation:

$$W_E e^{\alpha} = W_C e^{-\alpha} \Rightarrow e^{2\alpha} = \frac{W_C}{W_E}$$

So :

$$\alpha = \frac{1}{2} \log \left(\frac{W_C}{W_E} \right)$$

Justification for the Amount of Say

Given that, as previously observed, W_E is the total error and $W_C = 1 - W_E$, the previous equation can be rewritten as:

$$\alpha = \frac{1}{2} \log \left(\frac{1 - \text{TOTAL ERROR}}{\text{TOTAL ERROR}} \right)$$

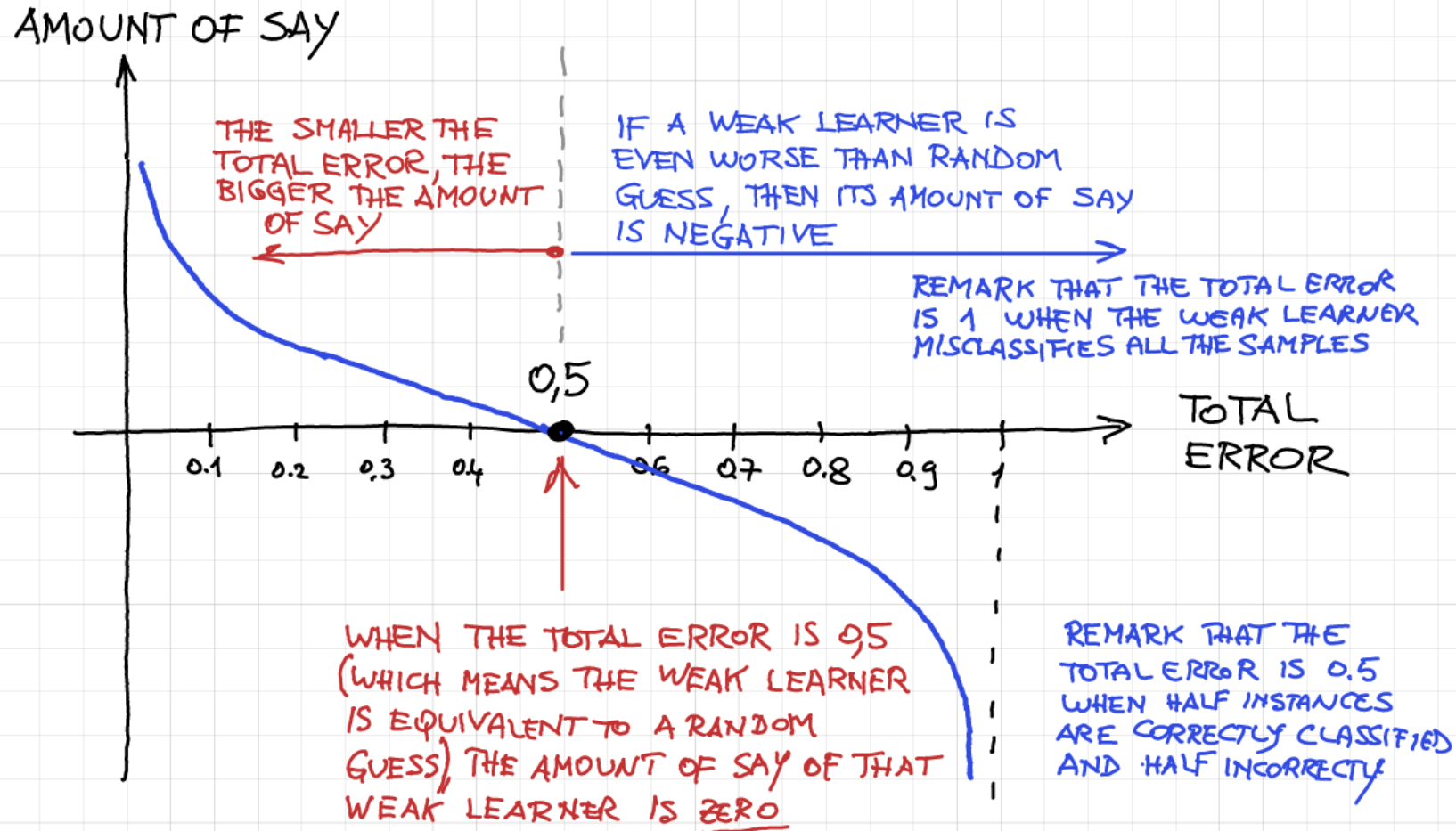
Reminder: this is the α that minimizes the exponential loss $\mathcal{L}$

Justification for the Amount of Say

Conclusion: the amount of say of a weak learner is determined in such a way that it minimizes the total exponential loss $\mathcal{L}$.

Informal Justification for the Amount of Say

$$\text{AMOUNT OF SAY} = \frac{1}{2} \log \left(\frac{1 - \text{TOTAL ERROR}}{\text{TOTAL ERROR}} \right)$$



Informal Justification for the Amount of Say

Remark: why do we give a negative amount of say to a learner that performs worse than random guess?

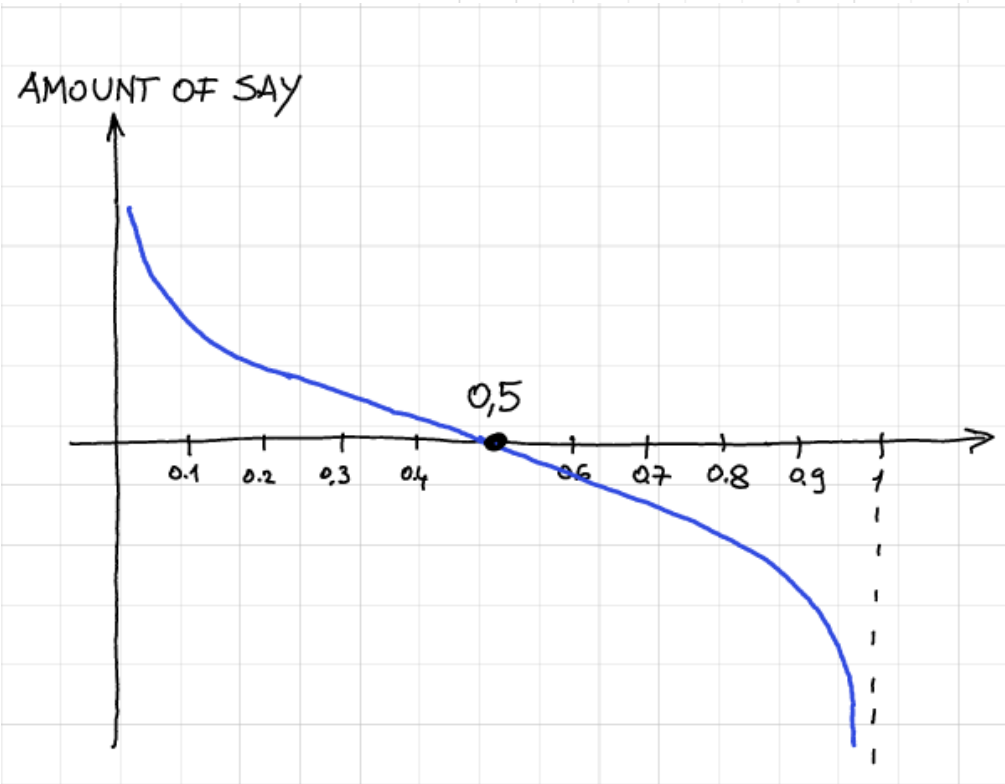
Because that learner has more incorrectly classified instances than correctly.

In other words, if that learner votes, say, for “Heart Disease”, it is probable that this prediction will be wrong **AND** giving that learner a negative amount of say will turn that vote into “**NOT** Heart Disease”!

Informal Justification for the Amount of Say

One more remark: let's have again a look at the function

$$\text{AMOUNT OF SAY} = \frac{1}{2} \log \left(\frac{1 - \text{TOTAL ERROR}}{\text{TOTAL ERROR}} \right)$$



From both the formula and the plot, we can see that if Total Error = 0 or Total Error = 1, then the function gives a mistake.

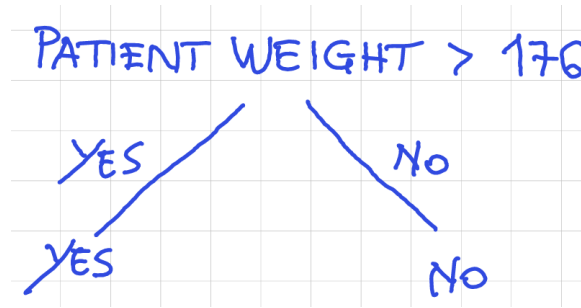
In practice, a small error term is added to prevent this from happening!

Why Exponential Loss?

- It gives an upper bound to the classification error
- It allows to update the sample weights (i.e., the weights of the observations) by means of simple calculation (and these updates lead to an exact minimization of the loss, and not an approximation of it)
- It is known to be a measure that encourages large margins, thus boosting the generalization ability of the algorithm.

Third Step: Determine The Amount of Say

So, let's go back to our first weak learner (let's call it WL_1):



We have seen that it misclassifies one sample, so its total error is $1/8$. So, it's amount of say is:

$$\text{AMOUNT OF SAY}(WL_1) = \frac{1}{2} \log \left(\frac{1 - \frac{1}{8}}{\frac{1}{8}} \right) = \underline{0.97}$$

Homework: calculate what would have been the amount of say of the other two candidate stumps if they had been chosen as the first weak learner of our ensemble.

Fourth Step: Update the Sample Weights

The next step consists in modifying the sample weights (i.e., the weights of the observations in the training set), so that the next weak learner will take into account the error(s) of the current weak learner.

Basically, what we need to do now is to emphasize the need for the next weak learner to correctly classify the observation(s) that was (were) incorrectly classified by the current weak learner.

Fourth Step: Update the Sample Weights

In other words, we will:

- Increase the sample weight of the sample(s) that had been misclassified by the current weak learner.
- Decrease the sample weight of all the other samples.

Fourth Step: Update the Sample Weights

The general formulas to update the sample weights are:

FOR INCORRECTLY CLASSIFIED INSTANCES : $SW_t = SW_{t-1} \cdot \exp(\text{AMOUNT OF SAY})$
FOR CORRECTLY CLASSIFIED INSTANCES : $SW_t = SW_{t-1} \cdot \exp(-\text{AMOUNT OF SAY})$

where sw_t is the new sample weight and sw_{t-1} is the old sample weight.

A justification for these formulas will be given shortly.

Fourth Step: Update the Sample Weights

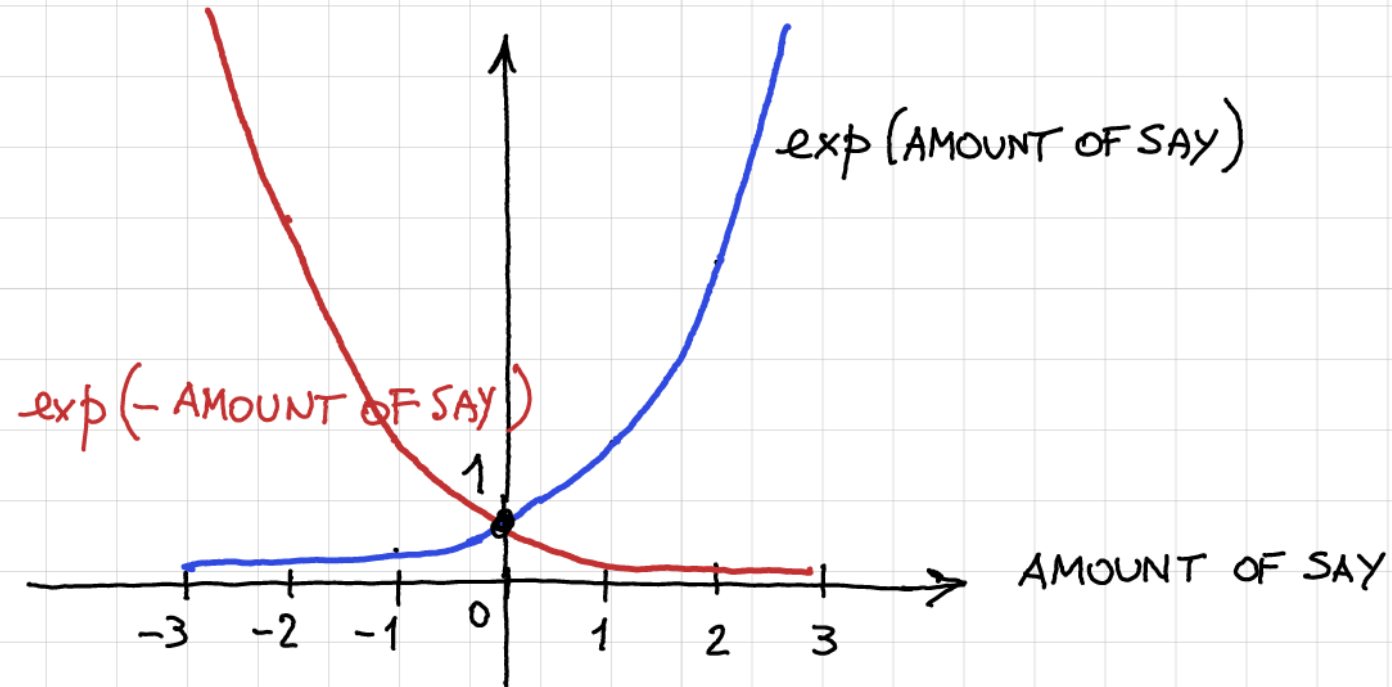
So, for instance, for the samples that were correctly classified by the first weak learner, we will have the following new sample weight:

$$1/8 \cdot \exp(-0.97) \sim 0.05$$

Motivation for the Sample Weights Update

FOR INCORRECTLY CLASSIFIED INSTANCES : $SW_t = SW_{t-1} \cdot \exp(\text{AMOUNT OF SAY})$

FOR CORRECTLY CLASSIFIED INSTANCES : $SW_t = SW_{t-1} \cdot \exp(-\text{AMOUNT OF SAY})$



Motivation for the Sample Weights Update

Watching the previous curves, we can understand that:

- For the incorrectly classified instances, if the amount of say is large (i.e., the weak learner did overall a good job) then we will scale the previous sample weight with a large number, otherwise a small number.
- For the correctly classified instances, if the amount of say is large then we scale the sample weight by a value close to zero (this will make the new sample weight very small), otherwise we scale the sample weight by a value close to 1 (this will make the new sample weight just a little smaller than the old one).

Observation

We have previously seen that the exponential loss can be rewritten as:

$$\mathcal{L} = \underbrace{\sum_{\text{CORRECT}} w_i \cdot e^{-\alpha}}_{\substack{\text{THIS IS THE NEW SAMPLE} \\ \text{WEIGHT FOR THE SAMPLES} \\ \text{THAT HAD PREVIOUSLY BEEN} \\ \text{CORRECTLY CLASSIFIED}}} + \underbrace{\sum_{\text{WRONG}} w_i \cdot e^{+\alpha}}_{\substack{\text{THIS IS THE NEW SAMPLE} \\ \text{WEIGHT FOR THE SAMPLES} \\ \text{THAT HAD PREVIOUSLY BEEN} \\ \text{INCORRECTLY CLASSIFIED}}}$$

So, the exponential loss is equal to the sum of the (unnormalized) sample weights of all the observations in the training set.

Remark: this equality holds only if the sample weights are **not** normalized (or, otherwise, normalizing also the loss, dividing its value by the total number of training observations).

Fourth Step: Update the Sample Weights

So, we can now conclude the fourth step by updating the sample weights. After calculating the new sample weights, we need to normalize them, so that they add up to 1.

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	OLD SAMPLE WEIGHT	NEW SAMPLE WEIGHT	NORMALIZED SAMPLE WEIGHT
YES	YES	205	YES	$1/8$	0.05	0.07
No	YES	180	YES	$1/8$	0.05	0.07
YES	No	210	YES	$1/8$	0.05	0.07
YES	YES	167	YES	$1/8$	0.33	0.49
No	YES	156	No	$1/8$	0.05	0.07
No	YES	125	No	$1/8$	0.05	0.07
YES	No	168	No	$1/8$	0.05	0.07
YES	YES	172	No	$1/8$	0.05	0.07

NOW WHEN WE ADD UP THE NEW SAMPLE WEIGHTS, WE GET 1 (PLUS OR MINUS A LITTLE ROUNDING ERROR).

BEFORE NORMALIZATION, SAMPLE WEIGHTS ADD UP TO 0.68, SO A GOOD WAY TO NORMALIZE THEM IS TO DIVIDE ALL OF THEM BY 0.68.

Now we can forget the previous sample weights. What we will use for the next weak learner is:

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	0.07
NO	YES	180	YES	0.07
YES	NO	210	YES	0.07
YES	YES	167	YES	0.49
NO	YES	156	NO	0.07
NO	YES	125	NO	0.07
YES	NO	168	NO	0.07
YES	YES	172	NO	0.07

Iteration of the algorithm

To choose the next weak learner so to take into account the new sample weights, we now have two possibilities:

1. Use the sample weights to calculate weighted GINI indexes (to determine which variable should be in the root of the next stump)
2. Create a new collection of samples that contains (or better, is likely to contain) duplicate copies of the samples with the largest sample weights.

Iteration of the algorithm

In this example, we assume we have chosen option number 2.

We start by creating a new, *empty*, dataset, that has the same size (same number of rows and same number of columns) as the original one.

Then, we pick up a random number R between 0 and 1 (continues in the next slide...)

Iteration of the algorithm

At this point:

- If $R \in [0, 0.07)$ then we insert the 1st sample in the new dataset
- If $R \in [0.07, 0.14)$ then we insert the 2nd sample in the new dataset
- If $R \in [0.14, 0.21)$ then we insert the 3rd sample in the new dataset
- If $R \in [0.21, 0.70)$ then we insert the 4th sample in the new dataset
- ...

and we repeat this process until the new dataset contains the same number of samples as the old one (of course, every time drawing a brand-new random number R).

Iteration of the algorithm

For instance, imagine that the random numbers we pick in the first 4 iterations are: 0.72, 0.42, 0.83, 0.51. Then, the samples we would copy in the new dataset would be the 5th, the 4th, the 6th and again the 4th.

And the construction of the new dataset until this point would look like this:

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
YES	YES	205	YES	0.07
NO	YES	180	YES	0.07
YES	NO	210	YES	0.07
YES	YES	167	YES	0.49
NO	YES	156	NO	0.07
NO	YES	125	NO	0.07
YES	NO	168	NO	0.07
YES	YES	172	NO	0.07

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE
NO	YES	156	NO
YES	YES	167	YES
NO	YES	125	NO
YES	YES	167	YES

Iteration of the algorithm

Notice that it is highly probable that in the new dataset the observations with the largest sample weight will appear in multiple copies.

When the new dataset is ready, we can get rid of the old one and use the new one.

At this point, we give all the samples equal sample weights, just like before (see next slide).

Iteration of the algorithm

CHEST PAIN	BLOCKED ARTERIES	PATIENT WEIGHT	HEART DISEASE	SAMPLE WEIGHT
NO	YES	156	NO	$1/8$
YES	YES	167	YES	$1/8$
NO	YES	125	NO	$1/8$
YES	YES	167	YES	$1/8$
YES	YES	167	YES	$1/8$
YES	YES	172	NO	$1/8$
YES	YES	205	YES	$1/8$
YES	YES	167	YES	$1/8$

It is worth stressing that giving all the samples equal sample weights doesn't mean that the next weak learner will not emphasize the need to correctly classify the samples that had been misclassified before.

In fact, those samples appear several times, creating in this way a large penalty for being misclassified.

Iteration of the algorithm

Now we are ready to iterate the algorithm, finding the stump that does the best job classifying the new collection of samples.

Termination Condition

Several options are possible:

- Fixed number of iterations. Terminate when a prefixed number T of weak learners has been created. This can be a good idea because AdaBoost may keep improving margins even after training error reaches zero, and this may be beneficial for limiting overfitting. In this way, T becomes a regularization parameter.
- Early stopping due to weak learner quality. We can terminate when no weak learner with a total error < 0.5 can be found. In particular, notice that if a weak learner has a total error $= 0.5$, its amount of say is zero and no progress is possible.
- Perfect classifier found. If we find a “weak” learner with total error $= 0$, we can decide to stop or not (see point 1.).

Besides these criteria, like for any other algorithm, early stopping given by a validation error not improving is possible.

Generalization Phase

Now we miss a final piece of information: how do we classify new (unseen) samples.

Let us assume, for instance, that when the algorithm terminates we have created 6 weak learners WL1, WL2, WL3, WL4, WL5 and WL6.

Let us assume that they have the following amount of say α :

$$\alpha(WL1) = 0.97, \alpha(WL2) = 0.32, \alpha(WL3) = 0.41, \\ \alpha(WL4) = 0.78, \alpha(WL5) = 0.82, \alpha(WL6) = 0.63$$

Generalization Phase

Now, let's assume that a new patient arrives. We evaluate all the weak learners on this new patient. Let us assume that:

- WL1, WL2, WL4 and WL6 classify the new patient as Heart Disease
- WL3 and WL5 classify the new patient as No Heart Disease

All we need to do is to add up the amount of say for the two groups of weak learners (the ones that classified as Heart Disease and the ones that classified as No Heart Disease).

Our final decision will be given by the largest of these two sums.

Generalization Phase

In our case, we have:

- $\alpha(\text{WT1}) + \alpha(\text{WT2}) + \alpha(\text{WT4}) + \alpha(\text{WT6}) = 0.97 + 0.32 + 0.78 + 0.63 = \mathbf{2.7}$
- $\alpha(\text{WT3}) + \alpha(\text{WT5}) = 0.41 + 0.82 = \mathbf{1.23}$

Conclusion: the new patient will be classified as Heart Disease.

Remark

If we turn the class labels (and the predictions) into +1 and -1, then the prediction on a sample x is obtained calculating:

$$\text{SNG} \left(\sum_{i=1}^{WL} \alpha_i h_i(x) \right)$$

where:

- WL is the number of weak learners
- α_i is the amount of say of the i^{th} weak learner
- $h_i(x)$ is the prediction of the i^{th} weak learner
- $\text{SNG}(y)$ is equal to +1 if $y \geq 0$, and it is equal to -1 otherwise.

Pseudocode of Adaboost

TRAINING PHASE :

- ① GIVE AN INITIAL SAMPLE WEIGHT = 1 TO ALL SAMPLES IN THE T.S.
- ② INITIALIZE THE ENSEMBLE E TO EMPTY AND SET $i = 1$
- ③ REPEAT UNTIL TERMINATION CONDITION
 - ③.1 NORMALIZE THE SAMPLE WEIGHTS SO THAT THEY SUM UP TO 1
 - ③.2 SELECT THE WEAK LEARNER WITH THE BEST WEIGHTED GINI INDEX FROM A SET OF POSSIBLE WEAK LEARNERS LET THAT WEAK LEARNER BE h_i
 - ③.3. CALCULATE THE TOTAL ERROR OF THE CHOSEN WEAK LEARNER h_i AS THE SUM OF THE (NORMALIZED) SAMPLE WEIGHTS OF THE SAMPLES IT MISCLASSIFIES
 - ③.4. CALCULATE THE AMOUNT OF SAY OF THAT WEAK LEARNER AS:
 $\alpha_i = 1/2 \cdot \log((1 - \text{TOTAL ERROR}) / \text{TOTAL ERROR})$
 - ③.5. INSERT THIS WEAK LEARNER h_i IN THE ENSEMBLE E
 - ③.6. FOR EVERY SAMPLE IN THE T.S. , UPDATE ITS SAMPLE WEIGHT (SW) AS : $\text{NEW SW} = \text{OLD SW} \cdot \exp(\alpha_i)$
 - ③.7. $i = i + 1$
- ④ RETURN THE ENSEMBLE E

GENERALIZATION PHASE : FOR EACH UNSEEN SAMPLE x , EVALUATE ALL THE WEAK LEARNERS IN THE ENSAMBLE ON x . THE FINAL CLASSIFICATION WILL BE GIVEN BY:

$$\text{SIG} \left(\sum_{i=1}^{\#E} \alpha_i h_i(x) \right)$$